

DIODEGUARD BLUEPRINT

AI-Driven Passive Cyber Threat Detection Pipeline

PROJECT NAME	VERSION	DOMAIN	TARGET HACKATHON
DiodeGuard	1.0.0	Cybersecurity / CNI	SIH 2026

1. PROJECT OVERVIEW

One-Line Summary

A real-time AI/ML pipeline that ingests one-directional IP traffic from a hardware data diode and detects, classifies, and scores cybersecurity threats using only passively collected metadata — with zero payload decryption and zero return path.

2. PROBLEM CONTEXT

2.1 What is a Data Diode?

A **hardware data diode** is a physically enforced one-way network bridge. It mirrors all traffic flowing across a gateway/peering link into a separate monitoring enclave. The critical property is:

- Traffic flows **IN** to the enclave → **✓ Allowed**
- Traffic flows **OUT** from the enclave → **✗ Physically Impossible**

This ensures the monitoring system can observe everything crossing the link, but cannot send probes, complete handshakes, push mitigation commands, or communicate back in any way.

2.2 Why Does This Matter?

Critical Infrastructure Operators (Power Grids, Telecom, Defense Networks, SCADA/ICS) use data diodes to ensure that even if the monitoring/analytics system is fully compromised by an attacker, they have **zero lateral movement** path back into the production network.

2.3 The Challenge

Build an AI pipeline that works **entirely within these constraints** — detecting threats from metadata alone, in real-time, without ever touching payloads or sending a single packet back.

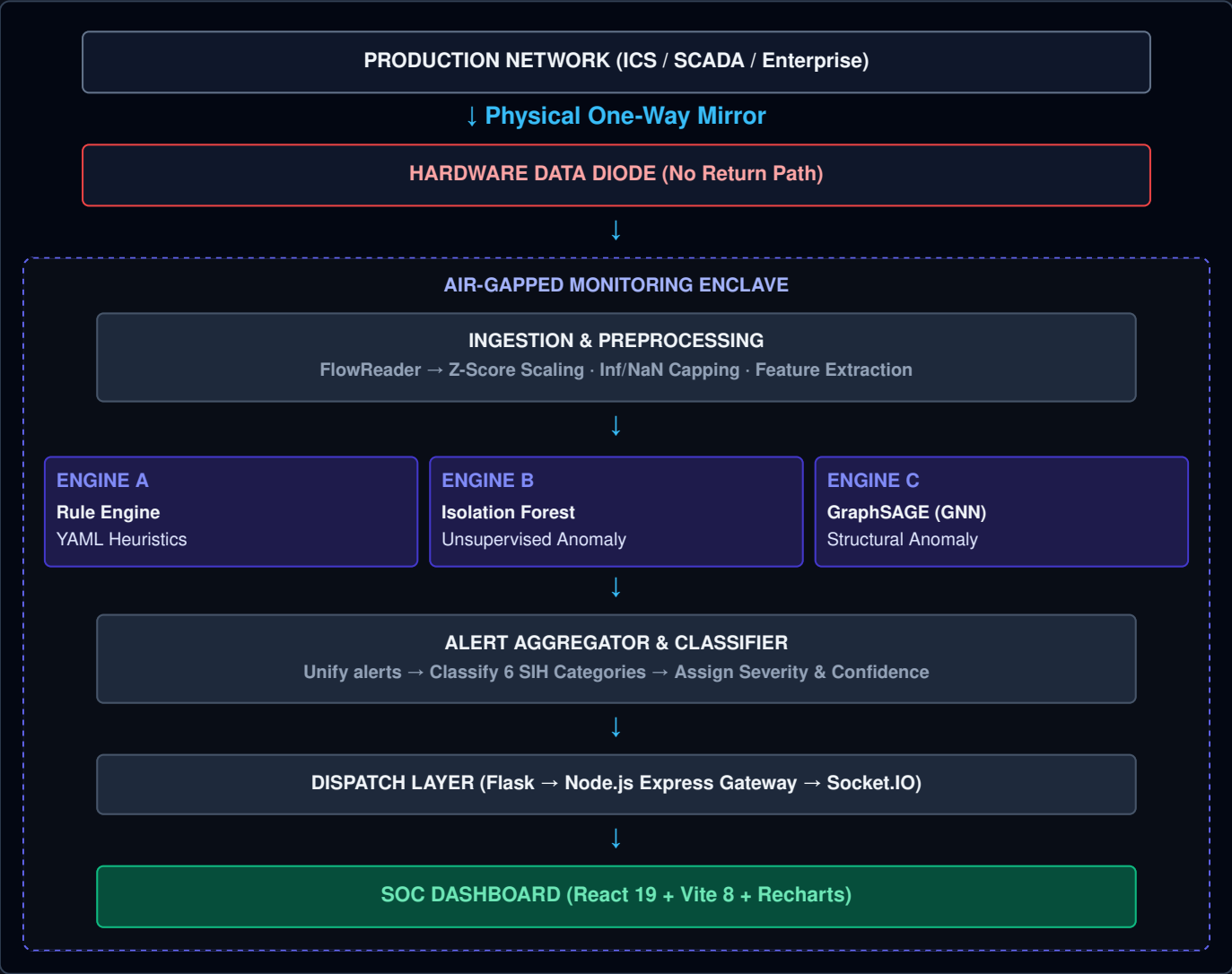
3. THREAT CATEGORIES (6 SIH-MANDATED)

#	THREAT TYPE	DETECTION METHOD USED	ENGINE
1	Volumetric / Protocol DDoS SYN floods, UDP reflection, spoofed floods	Flow-level rate thresholds, Flow Packets/s , Flow Bytes/s , source-IP entropy	RULE ENGINE
2	Botnet C2 Beaconing Periodic callbacks to C2 servers	Flow IAT Mean/Std periodicity analysis, repetitive flow duration patterns	RULE ENGINE + GRAPHSAGE
3			

#	THREAT TYPE	DETECTION METHOD USED	ENGINE
	DGA Domains & DNS Tunnelling Entropy/n-gram anomalies in DNS	DNS port (53) flow behavioral analysis, query length anomalies via flow metadata	RULE ENGINE
4	Malware in Encrypted Sessions TLS/QUIC metadata analysis	Packet-size sequences, Fwd/Bwd Packet Length distributions, timing patterns — NO decryption	RULE ENGINE + ISOFOREST
5	Reconnaissance & Port Scanning Fan-out from single source	Rapid small-packet flows to diverse destination ports, fan-out ratio detection	RULE ENGINE
6	Data Exfiltration Asymmetric outbound volume	Down/Up Ratio anomalies, unusual outbound-to-inbound byte ratios	RULE ENGINE + ISOFOREST

4. SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram



4.2 Technology Stack

LAYER	TECHNOLOGY	PURPOSE
ML Framework	PyTorch + PyTorch Geometric	GraphSAGE Graph Neural Network
ML Library	Scikit-Learn	Isolation Forest unsupervised anomaly detection
Rule Engine	Custom Python + YAML	Heuristic signature-based threat matching
Backend API	Python Flask	ML inference orchestration, REST endpoints
Gateway	Node.js + Express	WebSocket relay, Socket.IO broadcast
Frontend	React 19 + Vite 8	SOC Dashboard SPA
Charts & Animations	Recharts + Framer Motion	Real-time visual graphs & UI transitions
Styling & Monitoring	Tailwind CSS + psutil	Dark cyber-neon theme & System tracking

5. MODULE BREAKDOWN

5.1 Ingestion Module (backend/app/ingestion/)

- **FlowReader** (`flow_reader.py`): Reads the CSV dataset row-by-row as a Python generator, simulating a real-time stream. Each row is mapped to a `FlowRecord` dataclass containing: `src_ip` , `dst_ip` , `src_port` , `dst_port` , `protocol` , `duration` , forward/backward packet counts, byte counts, IAT statistics, and a `raw_features` dictionary containing all 77 numeric feature columns.
- **FlowRecord Schema** (`schemas/flow.py`): Dataclass defining canonical flow representations.

5.2 Rule Engine (backend/app/detection/rules/)

RuleEngine (`rule_engine.py`) loads YAML rule files from `rules/` at startup. On each flow, it evaluates all rule conditions and returns triggered rule names. Includes 7 active YAML rules:

- `port_scan.yaml` — Rapid fan-out to ports
- `ddos_volume.yaml` — Volumetric flood patterns
- `c2_beacon.yaml` — Periodic beaconing
- `dns_tunnel.yaml` — DNS port 53 anomalies

- `data_exfil.yaml` — Asymmetric outbound ratios
- `encrypted_malware.yaml` — Size distributions
- `generic_anomaly.yaml` — Catch-all heuristics

5.3 Isolation Forest & 5.4 GraphSAGE GNN

- **Isolation Forest AnomalyDetector**: Trained dynamically on the first 2,000 flows (assumed benign baseline) with `contamination=0.05` . Predicts inlier vs outlier per flow feature vector.
- **GraphSAGE Neural Network**: 2-layer `SAGEConv` network (`input_dim → 128 → 2`). Dynamic K-NN (K=3) graph construction attaches new flow nodes to a rolling window of 50 background flows. Triggers `STRUCTURAL_ANOMALY` if `P(malicious) > 0.60` .

5.5 Orchestrator & 5.6 Flask API Endpoints

Orchestrator passes incoming flows through all 3 engines concurrently, records latency, classifies alerts, tracks IP/protocol statistics, and posts payloads to Node.js.

ENDPOINT	METHOD	DESCRIPTION
/health	GET	System uptime, CPU, memory, engine status
/metrics	GET	Overview metrics: total flows, alerts, threat distribution
/replay	POST	Start replay stream {count: N, delay: D}
/alerts	GET	Alert history with severity & category filters
/traffic	GET	Traffic analytics: protocols, top talkers, ports
/models	GET	Per-engine performance, latency & detection rate

6. STANDARDIZED ALERT SCHEMA (SIH REQUIREMENT)

Every alert produced by any engine is normalized into this standardized JSON structure:

```
{
  "id": "ALERT-1714567890123",
  "timestamp": "2026-08-29T14:30:00.000Z",
  "flow_id": "a3b2c1d4-e5f6-7890-abcd-ef1234567890",
  "threat_class": "STRUCTURAL_ANOMALY",
  "category": "Anomaly",
  "severity": "CRITICAL",
  "source_ip": "192.168.10.15",
  "dest_ip": "203.0.113.5",
  "src_port": 49152,
  "dst_port": 443,
  "confidence": 95.8,
  "engine": "GraphSAGE",
  "evidence": "Anomalous graph structure detected in traffic network",
  "status": "NEW"
}
```

7. DATASET & FEATURE ENGINEERING

Dataset: CIC-IDS2017 (Canadian Institute for Cybersecurity) — [datasets/combinenew.csv](#) containing benign and attack flows (DDoS, Port Scan, Brute Force, Web Attack, Infiltration, Botnet).

CATEGORY	EXAMPLE FEATURES	PURPOSE
Volumetric	Flow Bytes/s , Flow Packets/s , Total Fwd Packets	DDoS / flood detection
Packet Size	Fwd Packet Length Max/Min/Mean , Bwd Packet Length Mean	Encrypted malware fingerprinting
Temporal/IAT	Flow IAT Mean/Std , Fwd IAT Mean , Bwd IAT Mean	C2 beaconing periodicity
Asymmetry	Down/Up Ratio , Subflow Fwd/Bwd Bytes	Data exfiltration detection
TCP Flags	SYN , FIN , PSH , ACK Flag Count	Protocol abuse detection
Bulk Transfer	Fwd Avg Bytes/Bulk , Bwd Avg Bulk Rate	Tunnelling anomalies

8. PROJECT STRUCTURE & QUICK START

Project Directory Layout

```
DiodeGuard/
├── backend/           # Flask API & ML Pipeline
│   └── app/
│       ├── main.py    # API Server (Port 8000)
│       ├── orchestrator.py
│       ├── ingestion/ # CSV Flow Streaming
│       └── detection/  # Rules, IsoForest, GNN
├── node_backend/      # Express + Socket.IO (Port 5000)
├── frontend/          # React 19 + Vite SOC Dashboard
└── datasets/          # CIC-IDS2017 CSV Data
```

Quick Start Execution

```
# Windows One-Click
run.bat

# Manual Setup:
cd backend && python app/main.py
cd node_backend && node server.js
cd frontend && npm run dev

# Dashboard: http://localhost:5173
```

9. SIH ARCHITECTURAL CONSTRAINTS COMPLIANCE

SIH CONSTRAINT	REQUIREMENT	DIODEGUARD IMPLEMENTATION	STATUS
Read-Only Ingest	No return path, zero inline blocking	FlowReader is a 1-way generator. Zero outbound calls to source.	✓ PASS
No Payload Decryption	TLS/QUIC analyzed via metadata only	Packet size distributions & byte ratios used. Zero DPI.	✓ PASS
Streaming Processing	Bounded latency, incremental processing	Per-flow inference with <5ms average engine latency.	✓ PASS
Defined Throughput	State and demonstrate traffic rate	Dashboard displays live throughput_fps (1,500+ flows/sec).	✓ PASS
Standardized Alerts	Structured record with flow ID, confidence, evidence	Complete standardized JSON alert schema implemented.	✓ PASS